Crecimiento exponencial: 2^n amplitudes

Genesis Silesky

20 de octubre de 2026

1. Crecimiento exponencial: 2^n amplitudes

1.1. Objetivo

Cuantificar por qué simular n qubits en una computadora clásica requiere memoria que crece como 2^n.

1.2. Conceptos previos

Concepto previo: producto tensorial y potencias de 2

Ver sección **??**. Idea clave: al combinar sistemas con $\otimes$, las dimensiones se *multiplican*: $2 \cdot 2 \cdots 2 = 2^n$. Además, $1\,\text{GiB} = 2^{30}$ bytes.

1.3. Fundamento teórico

El espacio de n qubits es $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$; un estado general es

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} c_x \, |x\rangle \,, \qquad \sum_x |c_x|^2 = 1, \tag{1}$$

con 2^n amplitudes complejas independientes (salvo normalización y fase global). Con `complex128` cada amplitud ocupa 16 bytes, de modo que

$$\text{memoria} = 16 \cdot 2^n \text{ bytes}. \tag{2}$$

Para $n = 30$: $16 \cdot 2^{30}$ B $= 16$ GiB; para $n = 50$: $16 \cdot 2^{50}$ B $= 2^{24}$ GiB $= 16$ PiB, y hay $\approx 1.13 \times 10^{15}$ amplitudes.

Un caso concreto. Aplicar H a cada uno de n qubits partiendo de $|0 \cdots 0\rangle$ da $H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$. Con $n = 4$: 16 amplitudes iguales a $1/4$.

Matiz importante

Esto aplica a la simulación de circuitos *generales* con el vector de estado. Hay circuitos con estructura (p. ej. solo puertas Clifford, o con poco entrelazamiento) que se simulan eficientemente con otros métodos. Tampoco significa que un computador cuántico "guarde" 2^n números accesibles: al medir solo se obtienen n bits.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 03 - Con n qubits hay 2^n amplitudes

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# %% Celda 2: memoria necesaria para simular n qubits
# Un estado de n qubits tiene 2^n amplitudes complejas.
# Cada complejo de doble precision (complex128) ocupa 16 bytes
# (8 para la parte real y 8 para la imaginaria).
for n in [1, 2, 10, 20, 30, 40, 50]:
    amplitudes = 2 ** n                 # numero de amplitudes
    gib = amplitudes * 16 / 2 ** 30     # bytes -> GiB (1 GiB = 2^30 bytes)
    print(f"n={n:>2}  amplitudes={amplitudes:>18,}  memoria ~ {gib:>14,.4f} GiB")
# Con 30 qubits ya son 16 GiB (una laptop buena); con 50, millones de GiB.
# Este crecimiento exponencial es la razon por la que simular es inviable
# a gran escala, y una de las motivaciones de la computacion cuantica.

# %% Celda 3: ver las 2^n amplitudes de un caso pequeno
n = 4
qc = QuantumCircuit(n)
# qc.h(range(n)) aplica Hadamard a cada uno de los n qubits.
# El resultado es la superposicion uniforme de las 2^n cadenas de bits.
qc.h(range(n))
sv = Statevector(qc)  # calcula el vector de estado del circuito
# Todas las amplitudes valen 1/sqrt(2^n) = 1/4 cuando n = 4.
print(f"{len(sv.data)} amplitudes, cada una = {sv.data[0].real:.4f}")
```

1.5. Explicación celda por celda

Celda 2 Recorre varios n, calcula 2^n y convierte a GiB dividiendo entre 2^{30}. Los formatos `:>18,` alinean y ponen separadores de miles.

Celda 3 Construye el estado $H^{\otimes 4}\,|0000\rangle$ y comprueba que hay 16 amplitudes, todas iguales a 0.25.

1.6. Salida obtenida

```
n= 1  amplitudes=                 2  memoria ~         0.0000 GiB
n= 2  amplitudes=                 4  memoria ~         0.0000 GiB
n=10  amplitudes=             1,024  memoria ~         0.0000 GiB
n=20  amplitudes=         1,048,576  memoria ~         0.0156 GiB
n=30  amplitudes=     1,073,741,824  memoria ~        16.0000 GiB
n=40  amplitudes= 1,099,511,627,776  memoria ~    16,384.0000 GiB
n=50  amplitudes=1,125,899,906,842,624  memoria ~ 16,777,216.0000 GiB
16 amplitudes, cada una = 0.2500
```

1.7. Interpretación

Cada qubit extra *duplica* la memoria. A 30 qubits ya se necesitan 16 GiB; unos pocos más y ninguna laptop alcanza. Este es el motivo de que simular sea difícil y, a la vez, parte de la motivación de usar hardware cuántico.

1.8. Ejercicios

1. ¿Cuántos qubits se pueden simular con 64 GiB de RAM?

2. Repita el cálculo con `complex64` (8 bytes por amplitud).

3. Verifique con `Statevector` que para $n = 3$ las amplitudes valen $1/\sqrt{8} \approx 0.354$.